



UNIVERSITY OF
GEORGIA

Final Project

04/24/2025
ECSE 4235

Group 7: Nathan Park, Umair Irshad

Final Project.....	0
1. Project Description.....	2
2. System Design.....	2
Overall Design.....	2
User Interface.....	3
MIDI File Reading.....	4
Speaker Output.....	6
3. Video.....	6

Github Link: [Here](#)

Roles:

- Umair Irshad: Software/Hardware/Documentation
- Nathan Park: Software/Hardware/Documentation

1. Project Description

The project is a **MIDI Music Player** that reads MIDI files and allows the user to select from the currently downloaded files to play. The project will be written in C with assembly.

- Assembly will be used to generate/stop the note waveforms at an input frequency.
- C will be used to handle the user interface, file reading/parsing, and timing.

Each MIDI file is a song that the user can select from and is stored onto the RP4 beforehand. The list of MIDI files is read and generated rather than hardcoded.

The current features for this project are:

- An adequate user interface that can choose from multiple MIDI files
- The ability to play and stop music at an adjustable volume and adjustable playback speed.

Regarding the required hardware, an LCD and rotary encoder is attached to the RP4. The LCD displays the current menu option alongside the currently selected song and the rotary encoder will allow the user to navigate the user interface. A single speaker will also be connected to the RP4. It will serve as the output for all of the music and due to the capabilities of MIDI files, we will be able to play multiple notes simultaneously. Any speaker can be used.

2. System Design

Overall Design

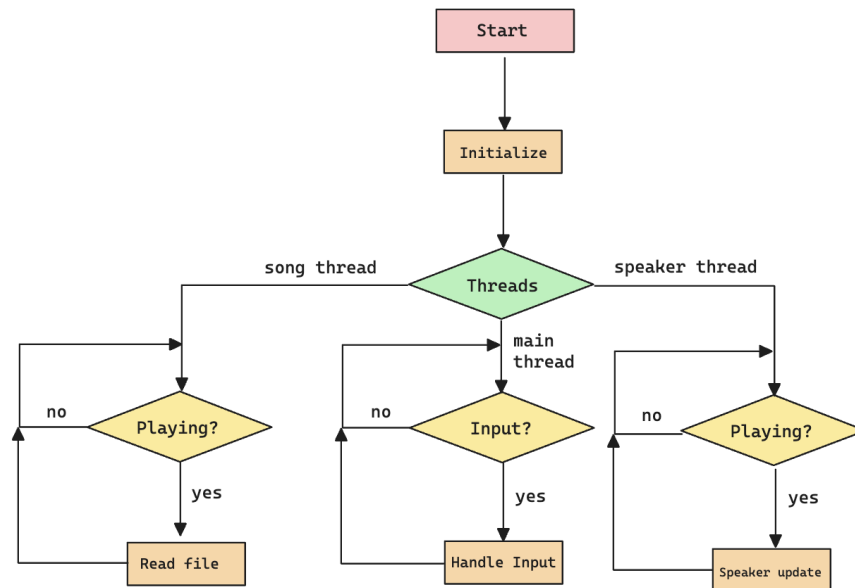


Figure 2.1: Flowchart of Main Program

The main program creates three threads — main thread (input), song thread, and speaker thread. This is due to the fact that the user must be able to interact with the interface even while other operations are ongoing.

- The main thread allows the user to interact using the rotary encoder and to utilize the user interface to set flags (playing) and change parameters (volume, playback speed, filename).
 - The song thread and speaker thread both start and continue operation based on the `playing` flag.
- The song thread deals with reading the selected file and sleeping the required amounts in between instructions (MIDI events). While reading the file, the song thread makes changes to the active notes list.
- The speaker thread constantly checks against the current time whether the GPIO should be outputting a HIGH or LOW based on the active notes list.

This logic is in `main.c`, which is the driver of the project.

User Interface

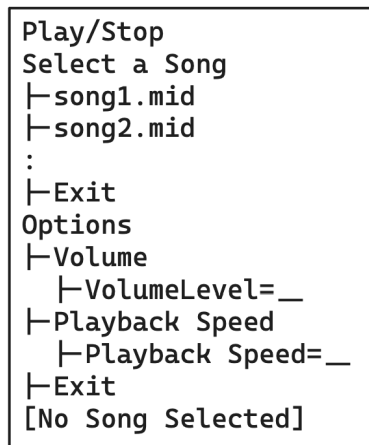


Figure 2.2: User Interface Plan

Regarding the user interface, there will be 3 options in the main branch: “Play/Stop”, “Select a Song”, and “Options”, as shown in Figure 2.1. Every option is its own screen (only showing the option that is hovered).

- “Play/Stop” will only show “Play” if there is nothing currently playing, and will only show “Stop” if there is a song playing. Selecting “Play” will start playing the currently selected MIDI file, and selecting “Stop” will stop playing the currently playing file.
- “Select a Song” will open the MIDI file navigation menu where the user can choose a MIDI file from the list of downloaded files.
- “Options” will open the options menu, where users can choose which options to modify between volume and playback speed.
- At the bottom of every screen, it will show the currently selected song, or “[No Song Selected]” if a song has not yet been selected.
- Selecting “Exit” will return the user to the main branch.

The program `user_interface.c` incorporates this design, and `parameters.c` manages the parameters modified within the user interface (volume and playback speed).

The user can navigate the interface using the rotary encoder. Rotating clockwise will move down the options and counterclockwise will move up the options. Pressing the encoder will select the current option and move deeper into the user interface if possible. The logic for the rotary encoder is implemented in `input_handler.c`.

The LCD utilizes the C library of `wiringPi` for I2C communication. However, a custom driver was needed to write to the LCD with functions such as `lcd_init()`, `lcd_set_cursor()`, `lcd_set_color()`, `lcd_print()`, and `lcd_clear()`.

MIDI File Reading

Upon selecting “Select a Song”, the user interface will show the list of available songs. The steps for this is as follows

1. The program opens the songs directory using the `dirent.h` library.
2. A circular doubly linked list will initialize and add all the files with the extension “.mid”.
 - If a list already exists, it is cleared rather than initialized.
3. Upon user input to cycle, the list will cycle in the same direction. Because it is a doubly linked list, both directions will work. Additionally, because it is a circular list, this can be done infinitely.
4. In order to exit the list, the program keeps track of the first and last songs.
 - “Exit” will show upon cycling forward on the last song or backward on the first song.

This logic is implemented in `file_manager.c` and `linked_list.c`.

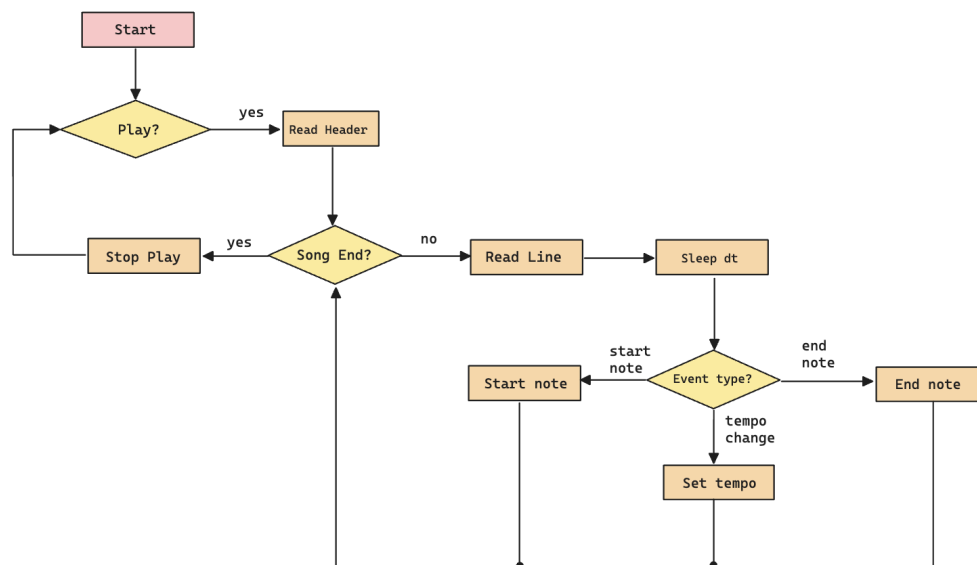


Figure 2.2: Flowchart for Handling Music Playing

Figure 2.2 shows the flowchart for the finalized design for handling music playing.

- The notes are played as the file is read.
 - Rather than storing the notes in a data structure, single track MIDI files can be played as the file is read, allowing for a much more efficient design.
- Once “Play” has been selected, the file is opened and the header is read.
 - The header contains valuable information such as the format, number of tracks, and division.
- If at the end of the file (no more notes) or the user has selected “Stop”, “Song End?” will result in True.
 - In either case, the program will return to waiting for input.
- The first byte of each “line” is the delta time in ticks.
 - The MIDI file specifies the division, which represents the amount of ticks per quarter note.
 - The default tempo (microseconds per quarter note) is 500,000. The MIDI file can send the tempo change meta event to change this tempo value.
 - The actual time in microseconds to sleep can be calculated as:

$$\text{Actual time } (\mu\text{s}) = \frac{\text{dt (ticks)} \times \text{Tempo } (\mu\text{s} / \text{quarter note})}{\text{Division (ticks / quarter note)}}$$

- The playback speed parameter is used here as a coefficient to the actual time.
- “Event type?” will read the event type of the file only.
 - Valid event types (1 byte) are: start note (1000 XXXX), end note (1001 XXXX), and meta events (1111 1111)
 - For start and end note: the next bytes after the event type is the note value (1 byte) and velocity (1 byte). Only the note value is considered for this project.
 - For meta events: the next bytes after the event type are the meta type (1 byte) and length of the meta instruction (1 byte). Only the tempo change meta event (01010001) is implemented, and the rest of the meta types can be skipped using the read length.
 - The rest of the event types are not needed, but need to be handled, as the size of the event instruction varies. The three unneeded event types are control change, program change, and channel aftertouch, which require an additional 2 bytes, 1 byte, and 2 bytes respectively.
- “Start note” will signal to the speaker to start playing the specified note.
 - Multiple notes can be played simultaneously.
- “End note” will signal to the speaker to stop playing the specified note.
 - If yes, then the waveform will end at the desired frequency. Then, it will dequeue this instruction to end the note and repeat the check for the next front of the queue.

Notes about this design:

- Will only support MIDI Format 0 (single track) files
- The time increment size (delta) can be changed, but will probably be set to 1.
- Subject to change depending on the development of the MIDI file reader.
 - There are more types of instructions that can be in a MIDI track than note start and end.

This logic is implemented in `song_controller.c`.

Speaker Output

Due to the presence of multiple notes, a standard PWM could not be used. Instead, the speaker thread is constantly calling an update function to check whether the GPIO should output a HIGH or LOW.

- The speaker controller manages and stores the active notes list. This list is of length 16, which means that 16 notes are permitted to play at a given time.
- When adding or removing a note to/from the list, the frequency must be calculated given the encoded byte (note) from the MIDI file. To calculate this, a function was made (in assembly) that utilizes floating point, double precision numbers.
- In order to account for multiple notes at once, a custom function (called on a loop) was made to generate a combined digital audio waveform by superimposing multiple square waves, each corresponding to an active note frequency.
 - Its goal is to compute whether the output signal should be HIGH or LOW at a given point in time based on all currently active notes.
 - It iterates through all active frequencies and determines the period and the current phase of each wave. It can calculate whether each square wave is HIGH or LOW based on the current phase relative to the period.
 - These individual outputs are then XORed together to produce a final composite waveform output.
 - This function was also done in assembly.
- Finally, to account for volume control, the current “tick” of the function (0-99) is compared against the volume (0-100). If the tick is greater than the volume, the output is set to LOW.
 - This essentially blocks a percentage of the actual outputs, therefore simulating volume.

This logic is done in `speaker_controller.c`, `get_frequency.s`, and `check_square.s`.

3. Video

DEMO VIDEO : [HERE](#)